

Socket-Go Messaging Protocol

Suphachock Kamlek 6710451321

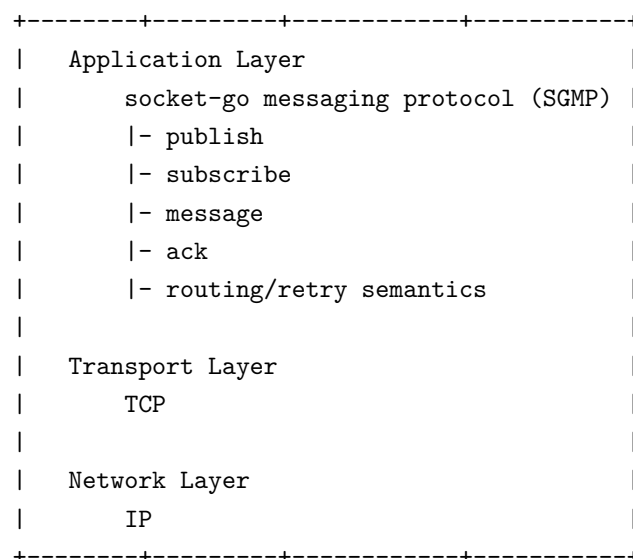
30 สิงหาคม พ.ศ. 2569

บทนำ

เอกสารนี้ระบุถึง Socket-Go Messaging Protocol (SGMP) ซึ่งเป็น application-layer messaging protocol บน TCP

SGMP มีระบบรับส่งข้อความแบบแบ่งตามหัวข้อ (Topic-based Publish/Subscribe), การรวมช่องสัญญาณเชิงตรรกะ (Logical Channel Multiplexing), การตอบรับข้อความ (Acknowledgement), การส่งซ้ำอัตโนมัติ (Retry), การจำกัดขนาดคิว (Bounded Queues) ตลอดจนการรับประกันการส่งข้อความอย่างน้อยหนึ่งครั้ง (At-least-once Delivery Semantics)

ตำแหน่งของ Protocol ใน Network Stack



การแก้ปัญหาของ Protocol

Protocol นี้เป็น messaging protocol ระหว่าง client กับ server โดยรองรับ

- publish message เข้า topic
- subscribe topic
- ส่ง message ให้ consumer
- ACK หลังประมวลผลเสร็จ
- retry เมื่อไม่มี ACK
- multiplex หลาย logical channel บน TCP connection เดียว
- backpressure ผ่าน bounded queue และ max in-flight

ระดับ Frame

Frame เป็นหน่วยข้อมูลระดับต่ำสุดบน TCP ใน protocol นี้จะมีรูปแบบ

+-----+-----+-----+-----+			
Type	Channel	Body Length	Body
1 byte	1 byte	4 bytes	N bytes
+-----+-----+-----+-----+			

รายละเอียด

- **Type** - ชนิดของ frame
- **Channel** - logical channel identifier
- **Body Length** - ขนาดของ body
- **Body** - payload ของ frame

Body ของแต่ละ frame จำกัดไม่เกิน 1 KiB แต่ logical message สามารถประกอบจากหลาย Body frame ได้

ระดับ Logical Message

หนึ่ง logical message จะมีรูปแบบ

- Header
- Method
- Body*
- End

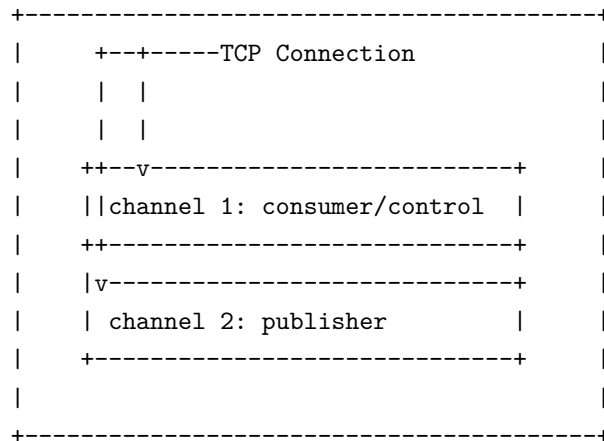
Body* หมายถึงมี 0 หรือหลาย body frame

ตัวอย่าง PUBLISH:

```
+-----+-----+
| Header:      |
| request-id=abc |
| topic=orders  |
|               |
| Method:      |
| PUBLISH      |
|               |
| Body:        |
| hello        |
|               |
| End:         |
| (empty)      |
|               |
+-----+-----+
```

Channel ใน Protocol

channel ใน protocol หมายถึง logical stream ภายใน TCP connection



channel รองรับ

- แยก publisher กับ consumer
- ระบุว่า ACK เกี่ยวข้องกับ channel ไหน
- รองรับหลาย logical operation ใน connection เดียว
- - ไม่ต้องเปิด TCP connection ใหม่ทุก role

Command ของ Protocol

Client ส่งไปที่ Server

- PUBLISH เพื่อ publish message ใน topic
- SUBSCRIBE เพื่อสมัครรับข้อมูลใน topic
- UNSUBSCRIBE ยกเลิกการรับข้อมูลใน topic
- ACK แจ้งการประมวลผล message สำเร็จ
- NACK แจ้งการประมวลผล message ไม่สำเร็จ
- TOPICS แสดงรายละเอียดจำนวน ชื่อ สถานะของ topic ทั้งหมดในระบบ ณ ปัจจุบัน
- PING ทดสอบการเชื่อมต่อ
- EXIT ยกเลิกการเชื่อมต่อ TCP และคืนทรัพยากร

ในแต่ละ command ควรระบุชื่อ command, channel ที่ใช้, header ที่จำเป็น, body ที่อนุญาต, response ที่คาดหวัง เช่นตัวอย่าง

```
+-----+-----+-----+-----+
|   PUBLISH   |
|   Required header: topic   |
|   Body: message payload   |
|   Channel: PublisherChannel   |
|   Response: PUBLISH_OK or ERROR   |
+-----+-----+-----+-----+
```

Flow ของ PUBLISH

PUBLISH_OK หมายถึง server รับ message เข้า queue แล้ว ไม่ได้หมายความว่า consumer ประมวลผลเสร็จแล้ว

```
+-----+-----+-----+-----+
|   Client   |
|   |
|   | PUBLISH(topic=orders, body=hello) |
|   v       |
|   Server Process   |
|   | validate channel is publisher type |
|   v       |
|   Exchange   |
|   | find topic (if not found return |
|   | ERROR: topic not found) |
|   v       |
|   Topic   |
|   | append message to pending queue |
|   v       |
|   response PUBLISH_OK to publisher |
+-----+-----+-----+-----+
```

Flow ของ MESSAGE และ ACK

หลังจากมี consumer subscribe topic

```
+-----+-----+-----+-----+
| Topic                                     |
| | choose consumer by round-robin algorithm |
| v                                       |
| MESSAGE                                |
| |                                     |
| v                                       |
| Consumer process                       |
| |                                     |
| v                                       |
| ACK                                    |
| |                                     |
| v                                       |
| Topic delete delivery from unacked list |
+-----+-----+-----+-----+
```

ตัวอย่าง MESSAGE metadata

```
+-----+-----+-----+-----+
| topic=orders                             |
| message-id=... (identify original message)|
| delivery-id=... (identify delivery order) |
| attempt=1                               |
+-----+-----+-----+-----+
```

ถ้าเกิดการ retry

```
+-----+-----+-----+-----+
| message-id=... (same)                   |
| delivery-id=... (new delivery id)       |
| attempt (increase)                      |
+-----+-----+-----+-----+
```

ดังนั้น consumer ต้องใช้ message id สำหรับ deduplication

Delivery Semantics

Protocol นี้ใช้ At-least-once delivery หมายความว่า message จะถูกส่งซ้ำได้ หาก

- consumer ไม่ส่ง ACK
- ACK timeout

- TCP send failure
- consumer disconnect
- consumer ส่ง NACK

Protocol นี้ไม่ได้รับประกัน exactly-once delivery เพราะการรับ message กับการรับ ACK อาจเกิดคนละช่วงเวลา

Max In-Flight

max in-flight คือจำนวน delivery ที่ส่งให้ consumer แล้วแต่ยังไม่มี ACK

```

+-----+-----+-----+-----+
|   maxInFlight = 1   |
|                     |
|   send A            |
|   waiting ACK A     |
|   ACK A             |
|   send B            |
+-----+-----+-----+-----+

```

แต่ถ้ายกตัวอย่าง max in-flight เป็น 32 แล้ว consumer จะมี message ที่ยังไม่ ACK ได้สูงสุด 32 รายการ นี่คือการไหลของ flow control ระดับ application

```

+-----+-----+-----+-----+-----+
|   pending queue → in-flight window → consumer   |
+-----+-----+-----+-----+-----+

```

Retry และ NACK

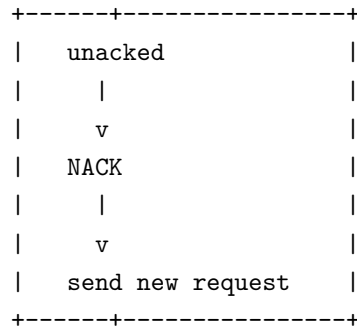
เมื่อ delivery timeout

```

+-----+-----+-----+-----+-----+
|   unacked   |
|   |         |
|   v         |
|   timeout   |
|   |         | |
|   |-- attempt does not exceed limit |
|   |         |-- requeue             |
|   |         |                         |
|   |-- over limit |
|         |-- drop                     |
+-----+-----+-----+-----+-----+

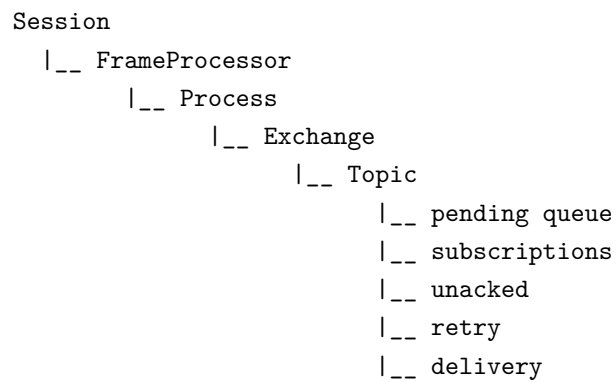
```

เมื่อ consumer ส่ง NACK



โครงสร้างภายใน server

สิ่งที่อยู่บน wire คือ protocol command แต่สิ่งที่อยู่ภายใน server คือ actor pipeline



ความรับผิดชอบ

- Session จัดการ TCP connection
- FrameProcessor ประกอบ frame เป็น logical message
- Process จัดการ channel ของ session
- Exchange จัดการ topic registry และ routing
- Topic จัดการ queue, delivery, ACK และ retry
- Channel logical delivery endpoint
- Client ส่ง command, รับ MESSAGE และส่ง ACK

ตัวอย่าง complete sequence

